

AARHUS UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE

COMPILATION

Hand-in 3 (Individual Report)

Author

Søren M. DAMSGAARD

Student number

202309814

September 24, 2026



Task 1 - Implementation of Semantic Analysis

The implementation of this heavily focuses on the module `semant.ml` and its functions `infer_type_expr` and `typecheck_expr`, these functions recursively call each other to traverse the AST translating the untyped AST into a typed AST, making sure to follow Dolphins type rules.

This implementation also uses the modules `env` and `errors` to handle variable scopes when encountering nested variables and handling errors/mismatched types.

Task 2 - Implementation of Code Generation

Generating code by translating the TypedAst to functional LLVM- code. is a huge part of this project, hence a separate module `codegen.ml` was created

This was done by using the CfgBuilder to create basic blocks, and allocating variables using `Ll.Alloca` with a the `cgenv` environment, that maps symbols to LLVM pointers.

In the code there is an explicit step when using a division or remainder op, it's a `Icmp Eq` check that safely exits if a divisor is zero.

Task 3 - Testing

The module `compiler.ml` takes an untyped AST program, sends it to the semantic analysis, and if the error list is empty it continues to the codegen module that returns a LLVM-program, and if an error is found the list of errors is printed and the compile is aborted.

To be extra safe, an entire test corpus was made, with A LOT of tests that covers everything, both trivial and more concrete features.

Questions

Question 1 - type Inference or type checking

Type inference is used when a variable declaration doesn't have an explicit type, and it has to be inferred from the bottom-up in the AST.

Type checking is when the semantics has strict rules, such as arithmetic operators (Plus, Minus) that needs the l/r operands to be Integers.

Combined, inference and checking supports each other well.

Question 2

The way we implemented error recovery was to make a dedicated `ErrorType` if an AST node had a mismatched type.

When the `typecheck_expr` encounters a mismatch it appends the error to an Error list and

returns the expression with a `TAst.ErrorType` continuing the analysis, and in the end it logs all errors/mismatch, making it easier to see what went wrong.

Question 3

*Glory - Didn't do it.

How to run the code

You will have received a zip file containing the complete compiler source code, for phase 1, the Dune build configurations, and the C runtime file.

This is run/compiled in VScode using the docker container provided.

To compile and run the project inside a VS Code development container, execute the following commands in the terminal while inside the folder:

1. `dune build`
2. `dolphin compile --from-ast --phase 1 tests/<test name>.json -o out.ll` (compiles the serialized AST and outputs an LLVM-ir file)
3. `clang runtime.c out.ll -o res`
4. `./res` (runs the program and outputs any intermediate prints in the terminal)
5. `echo $?` (outputs the return statement from the LLVM-ir code to the terminal)